

ECE 385

Fall 2025

Final Project Proposal

Krishnan Shankar (ks128@illinois.edu)
Aniketh Tarikonda (aniketh8@illinois.edu)

Contents

1	Overview	1
2	List of Features	1
2.1	Base Features	1
2.2	Advanced Features	1
3	Block Diagram	2
4	Expected Difficulty	2
5	Timeline	3

1 Overview

We propose to design and implement a comprehensive flight simulator, similar to Microsoft Flight Simulator and FlightGear (albeit not as complex), using the RealDigital Urbana Board. The flight simulator will present users with a virtual plane, which they can control using input devices (keyboard, joystick, etc.) to take off, fly, and land. The surroundings and heads-up display (HUD) will be rendered in real-time on a connected monitor, providing an immersive experience.

To harness the power of digital design, all flight dynamics calculations and graphics rendering will be implemented in SystemVerilog. Hardware-based implementations of physics calculations allows for more accurate simulations, since faster computation allows for smaller timesteps and thus more accurate simulations. Additionally, designing the GPU in hardware (hardware-accelerated graphics) is almost a fundamental requirement for a decent frame rate, which is crucial for a smooth user experience. Finally, the hardware design will be responsible for reading from an SD card (and interfacing with DDR memory), which is necessary for reading plane characteristics (weight, lift/drag coefficients, etc.) and environment/terrain information.

On the other hand, the MicroBlaze processor will mainly be used to handle user inputs, which will update the plane's state (throttle level, control surface deflections) through memory-mapped GPIO interfaces. The MicroBlaze will also deal with processing plane characteristics and environment data from the SD card (through DDR memory), which it will use to inform flight dynamics calculations and update the software framebuffer.

2 List of Features

2.1 Base Features

- Flight Control (Thrust, Ailerons, Elevators, Rudder)
- Accurate Flight Dynamics (Lift, Drag, Gravity, Wind, etc.)
- HUD (Artificial Horizon, Altitude, Airspeed, Vertical Speed, RPM, Heading)
- Basic Terrain (Flat Ground with Procedurally Generated Colors)
- Sound Effects

2.2 Advanced Features

- 3D Rendering for:
 - Terrain (Elevation and Realistic Depth)
 - Follow Camera (View from Behind the Plane)
 - Clouds/Weather Effects
- Advanced Flight Control (Flaps, Spoilers, Landing Gear, Brakes)
- Multiple Aircraft Models with Unique Flight Characteristics
- Airport Runways and Taxiways
- Flight Map + Flight Path

3 Block Diagram

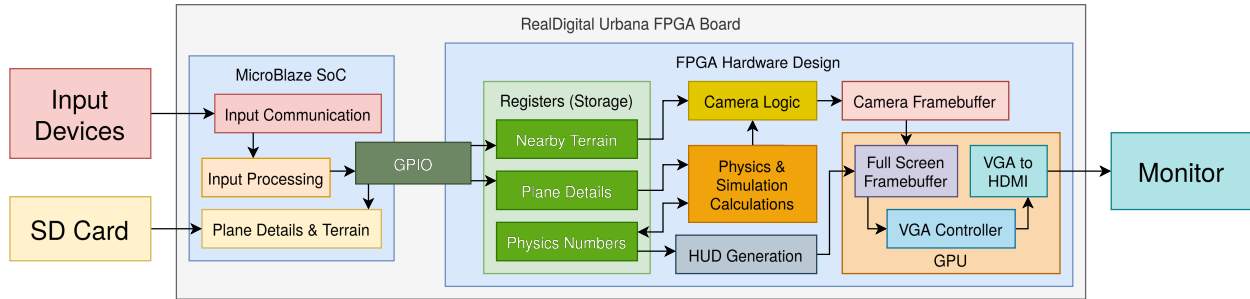


Figure 1: Block Diagram

4 Expected Difficulty

We expect the base featureset to be of relatively high difficulty (11/15), as it involves significant effort in implementing a graphics pipeline and real-world-accurate flight dynamics in hardware.

Being a 3D game, the graphics pipeline is much more complex than a simple 2D renderer for classic games—this means a lot more math and hardware logic is necessary to implement the same functionality, even with terrain that has no elevation changes. This, combined with the intricate physics calculations needed for realistic flight dynamics, makes the base featureset a challenging but achievable goal.

Specifically, the hardware design needs to:

- Read flight characteristics from GPIO
- Use complex physics equations to determine the plane's motion
- Update the plane's position and orientation for the next frame
- For each pixel:
 - Use the plane's position and orientation to determine the (x, y) coordinates on the ground that correspond to that pixel
 - Determine the color of that pixel based on the terrain data
 - Write the pixel color to the framebuffer

When looking at advanced features, we can assign an approximate difficulty to each major enhancement:

- 3D Rendering for Terrain would add approximately 2/15 difficulty points, as it significantly increases the complexity of the graphics pipeline. Instead of simply mapping pixels to colors based on an (x, y) coordinate, we now need to perform a form of ray tracing to determine which parts of the 3D terrain are visible from the plane's perspective. This is difficult!

- 3D Rendering for a Follow Camera would add approximately 3/15 difficulty points, and is a very challenging task. The main complexity arises from needing to render a 3D model of the plane itself, which is basically a full 3D object rendering problem on its own.
- 3D Rendering for Clouds/Weather Effects would add approximately 1/15 difficulty points if the follow camera is already implemented, since it now requires the 3D rendering engine to account for moving objects (and not just a plane that's stationary in the frame).
- Advanced Flight Control would add approximately 1/15 difficulty points, as it mainly involves a lot more complexity for physics calculations, but not much change to the graphics pipeline.
- Multiple Aircraft Models would add approximately 1/15 difficulty points, as it again involves complications to the physics calculations. However, if the follow camera is implemented, it would add another 1/15 difficulty points due to needing to render different 3D models.
- Implementing both Airport Runways and the Flight Map would add approximately 1/15 difficulty points in total, since they mostly involve UI and terrain generation changes that significantly impact user experience without being too complex to implement.

5 Timeline

Week 1:

- Write project proposal
- Implement SD card and DDR memory interfaces
- Implement basic GPU framework (framebuffer, VGA controller, VGA-to-HDMI)

Week 2:

- Implement input device communication and processing
- Implement physics engine and state registers (position, orientation, velocity, etc.)
- Implement HUD logic and rendering

Week 3:

- Implement GPIO interfaces
- Implement procedural terrain
- Implement camera logic using plane position/orientation and terrain data
- Integrate all components and perform initial testing

Week 4:

- Debug design
- Add advanced features as time permits